

Delegates and Events

- <http://etutorials.org/Programming/programming+microsoft+visual+c+sharp+2005/Part+III+More+C+Language/Chapter+8+Delegates+and+Events/Delegates/>

- Function pointers are prevalent in unmanaged programming languages (C or C++) and helpful in streamlining algorithms and choosing functions at run time rather than statically at compile time
- As function pointers, functions can be treated as local variables, function parameters, or even return values- which adds considerable flexibility in the way they are managed and called.

- Because pointers are simply pointers into raw memory, function pointers are not safe.
- .NET offers a type-safe version of function pointers called Delegates
- Delegates provide the same flexibility as function pointers but in a type-safe manner

- A delegate is an object that can refer to a method. Therefore, when we create a delegate, we are creating an object that can hold a reference to a method.
- Furthermore, the method can be called through this reference.
- In other words, a delegate can invoke the method to which it refers.

- It is important to understand that the same delegate can be used to call different methods during the runtime of a program by simply changing the method to which the delegate refers.
- Thus, the method that will be invoked by a delegate is not determined at compile time, but rather at runtime.
- This is the principal advantage of a delegate.

- *A delegate in C# is similar to a function pointer in C/C++.*

- A delegate type is declared using the keyword `delegate`.
- The general form of a delegate declaration is shown here:

`delegate ret-type name(parameter-list);`

- Here, ret-type is the type of value returned by the methods that the delegate will be calling.
- The name of the delegate is specified by name.
- The parameters required by the methods called through the delegate are specified in the parameter-list.
- Once created, a delegate instance can refer to and call methods whose return type and parameter list match those specified by the delegate declaration.

- A key point to understand is that a delegate can be used to call any method that agrees with its signature and return type.
- Furthermore, the method can be either an instance method associated with an object or a static method associated with a class.
- All that matters is that the return type and signature of the method agree with those of the delegate.

// A simple delegate example.

```
using System;
// Declare a delegate type.
delegate string StrMod(string str);
class DelegateTest {
// Replaces spaces with hyphens.
static string ReplaceSpaces(string s) {
Console.WriteLine("Replacing spaces with
    hyphens.");
return s.Replace(' ', '-');
}
```

```
// Remove spaces.  
static string RemoveSpaces(string s) {  
    string temp = "";  
    int i;  
    Console.WriteLine("Removing spaces.");  
    for(i=0; i < s.Length; i++)  
        if(s[i] != ' ') temp += s[i];  
    return temp;  
}
```

```
// Reverse a string.  
static string Reverse(string s) {  
    string temp = "";  
    int i, j;  
    Console.WriteLine("Reversing string.");  
    for(j=0, i=s.Length-1; i >= 0; i--, j++)  
        temp += s[i];  
    return temp;  
}
```

```
static void Main() {  
    // Construct a delegate.  
    StrMod strOp = new StrMod(ReplaceSpaces);  
    string str;  
    // Call methods through the delegate.  
    str = strOp("This is a test.");  
    Console.WriteLine("Resulting string: " + str);  
    Console.WriteLine();  
    strOp = new StrMod(RemoveSpaces);  
    str = strOp("This is a test.");  
    Console.WriteLine("Resulting string: " + str);  
    Console.WriteLine();  
    strOp = new StrMod(Reverse);  
    str = strOp("This is a test.");  
    Console.WriteLine("Resulting string: " + str);  
}  
}
```

- The output from the program is shown here:
- Replacing spaces with hyphens.
- Resulting string: This-is-a-test.
- Removing spaces.
- Resulting string: Thisisatest.
- Reversing string.
- Resulting string: .tset a si sihT

- Let's examine this program closely.
- The program declares a delegate type called StrMod, shown here:
- **delegate string StrMod(string str);**
- Notice that StrMod takes one string parameter and returns a string.

- Next, in `DelegateTest`, three static methods are declared, each with a single parameter of type `string` and a return type of `string`.
- Thus, they match the `StrMod` delegate.
- These methods perform some type of string modification.

- Notice that `ReplaceSpaces( )` uses one of string's methods, called `Replace( )`, to replace spaces with hyphens.
- In `Main( )`, a `StrMod` reference called `strOp` is created and assigned a reference to `ReplaceSpaces( )`.
- Pay close attention to this line:
- **`StrMod strOp = new StrMod(ReplaceSpaces);`**
- Notice how the method `ReplaceSpaces( )` is passed as a parameter.
- Only its name is used; no parameters are specified.

- This can be generalized.
- When instantiating a delegate, we specify only the name of the method to which we want the delegate to refer.
- Of course, the method's signature must match that of the delegate's declaration.
- If it doesn't, a compile time error will result.

- Next, **ReplaceSpaces()** is called through the delegate instance `strOp`, as shown here:
- **`str = strOp("This is a test.");`**
- Because `strOp` refers to `ReplaceSpaces( )`, `ReplaceSpaces( )` is invoked.

- Next, strOp is assigned a reference to RemoveSpaces(), and then strOp is called again.
- This time, RemoveSpaces() is invoked.
- Finally, strOp is assigned a reference to Reverse() and strOp is called.
- This results in Reverse() being called.

- The key point of the example is that the invocation of `strOp` results in a call to the method referred to by `strOp` at the time at which the invocation occurs.
- Thus, the method to call is resolved at runtime, not compile time.

Delegate Method Group Conversion

- C# has included an option that significantly simplifies the syntax that assigns a method to a delegate.
- This feature is called *method group conversion*, and it allows to simply assign the name of a method to a delegate, without using **new** or explicitly invoking the delegate's constructor.
- For example, here is the **Main()** method of the preceding program rewritten to use method group conversions:

- `static void Main() {`
- `// Construct a delegate using method group conversion.`
- `StrMod strOp = ReplaceSpaces; // use method group conversion`
- `string str;`
- `// Call methods through the delegate.`
- `str = strOp("This is a test.");`
- `Console.WriteLine("Resulting string: " + str);`
- `Console.WriteLine();`
- `strOp = RemoveSpaces; // use method group conversion`
- `str = strOp("This is a test.");`
- `Console.WriteLine("Resulting string: " + str);`
- `Console.WriteLine();`
- `strOp = Reverse; // use method group conversion`
- `str = strOp("This is a test.");`
- `Console.WriteLine("Resulting string: " + str);`
- `}`

- Notice the way the strOp is created and assigned the method ReplaceSpaces in this line:

StrMod strOp = ReplaceSpaces; // use method group conversion

- The name of the method is assigned directly to strOp.
- C# automatically provides a conversion from the method to the delegate type.
- This syntax can be generalized to any situation in which a method is assigned to (or converted to) a delegate type.

Using Instance Methods as Delegates

- Although the preceding example used static methods, a delegate can also refer to instance methods.
- It must do so, however, through an object reference.
- For example, here is a rewrite of the previous example, which encapsulates the string operations inside a class called **StringOps**.

```
// Delegates can refer to instance methods, too.
using System;
// Declare a delegate type.
delegate string StrMod(string str);
class StringOps {
// Replaces spaces with hyphens.
public string ReplaceSpaces(string s) {
Console.WriteLine("Replacing spaces with hyphens.");
return s.Replace(' ', '-');
}
```

```
// Remove spaces.  
public string RemoveSpaces(string s) {  
    string temp = "";  
    int i;  
    Console.WriteLine("Removing spaces.");  
    for(i=0; i < s.Length; i++)  
        if(s[i] != ' ') temp += s[i];  
    return temp;  
}
```

```
// Reverse a string.  
public string Reverse(string s) {  
    string temp = "";  
    int i, j;  
    Console.WriteLine("Reversing string.");  
    for(j=0, i=s.Length-1; i >= 0; i--, j++)  
        temp += s[i];  
    return temp;  
}  
}
```

```
class DelegateTest {
static void Main() {
StringOps so = new StringOps(); // create an instance of
    StringOps
// Initialize a delegate.
StrMod strOp = so.ReplaceSpaces;
string str;
// Call methods through delegates.
str = strOp("This is a test.");
Console.WriteLine("Resulting string: " + str);
Console.WriteLine();
}
```

```
strOp = so.RemoveSpaces;  
str = strOp("This is a test.");  
Console.WriteLine("Resulting string: " + str);  
Console.WriteLine();  
strOp = so.Reverse;  
str = strOp("This is a test.");  
Console.WriteLine("Resulting string: " + str);  
}  
}
```

- This program produces the same output as the first, but in this case, the delegate refers to methods on an instance of **StringOps**.

Multicasting

- One of the most exciting features of a delegate is its support for multicasting.
- Multicasting is the ability to create an invocation list, or chain, of methods that will be automatically called when a delegate is invoked.
- Such a chain is very easy to create.
- Simply instantiate a delegate, and then use the + or += operator to add methods to the chain.
- To remove a method, use – or – =.

- If the delegate returns a value, then the value returned by the last method in the list becomes the return value of the entire delegate invocation.
- Thus, a delegate that makes use of multicasting will often have a void return type.

Example of multicasting

- Notice that it reworks the preceding examples by changing the string manipulation method's return type to void and using a ref parameter to return the altered string to the caller.
- This makes the methods more appropriate for multicasting.

```
// Demonstrate multicasting.  
using System;  
// Declare a delegate type.  
delegate void StrMod(ref string str);  
class MultiCastDemo {  
    // Replaces spaces with hyphens.  
    static void ReplaceSpaces(ref string s) {  
        Console.WriteLine("Replacing spaces with hyphens.");  
        s = s.Replace(' ', '-');  
    }  
}
```

```
// Remove spaces.  
static void RemoveSpaces(ref string s) {  
    string temp = "";  
    int i;  
    Console.WriteLine("Removing spaces.");  
    for(i=0; i < s.Length; i++)  
        if(s[i] != ' ') temp += s[i];  
    s = temp;  
}
```

```
// Reverse a string.  
static void Reverse(ref string s) {  
    string temp = "";  
    int i, j;  
    Console.WriteLine("Reversing string.");  
    for(j=0, i=s.Length-1; i >= 0; i--, j++)  
        temp += s[i];  
    s = temp;  
}
```

```
static void Main() {  
    // Construct delegates.  
    StrMod strOp;  
    StrMod replaceSp = ReplaceSpaces;  
    StrMod removeSp = RemoveSpaces;  
    StrMod reverseStr = Reverse;  
    string str = "This is a test";  
    // Set up multicast.  
    strOp = replaceSp;  
    strOp += reverseStr;
```

```
// Call multicast.  
strOp(ref str);  
Console.WriteLine("Resulting string: " + str);  
Console.WriteLine();  
// Remove replace and add remove.  
strOp -= replaceSp;  
strOp += removeSp;  
str = "This is a test."; // reset string  
// Call multicast.  
strOp(ref str);  
Console.WriteLine("Resulting string: " + str);  
Console.WriteLine();  
}  
}
```


- Here is the output:
- Replacing spaces with hyphens.
- Reversing string.
- Resulting string: tset-a-si-sihT
- Reversing string.
- Removing spaces.
- Resulting string: .tsetasisihT

- In **Main()**, four delegate instances are created.
 - One, **strOp**, is null.
 - The other three refer to specific string modification methods.

- Next, a multicast is created that calls `RemoveSpaces( )` and `Reverse( )`.
- This is accomplished via the following lines:
`strOp = replaceSp;`
`strOp += reverseStr;`
- First, `strOp` is assigned `replaceSp`.
- Next, using `+=`, `reverseStr` is added.
- When `strOp` is invoked, both methods are invoked, replacing spaces with hyphens and reversing the string, as the output illustrates.

- Next, `replaceSp` is removed from the chain, using this line:
`strOp -= replaceSp;`
- and `removeSP` is added using this line:
`strOp += removeSp;`
- Then, `strOp` is again invoked.
- This time, spaces are removed and the string is reversed.
- Delegate chains are a powerful mechanism because they allow you to define a set of methods that can be executed as a unit.
- This can increase the structure of some types of code.

Covariance and Contravariance

- There are two features that add flexibility to delegates:
 - *covariance and contravariance.*
- *Normally*, the method that we pass to a delegate must have the same return type and signature as the delegate.
- However, covariance and contravariance relax this rule slightly, as it pertains to derived types.
- Covariance enables a method to be assigned to a delegate when the method's return type is a class derived from the class specified by the return type of the delegate.
- Contravariance enables a method to be assigned to a delegate when a method's parameter type is a base class of the class specified by the delegate's declaration.

// Demonstrate covariance and contravariance.

using System;

```
class X {  
    public int Val;  
}
```

// Y is derived from X.

```
class Y : X { }
```

// This delegate returns X and takes a Y argument.

```
delegate X Changelt(Y obj);
```

```
class CoContraVariance {
```

// This method returns X and has an X parameter.

- static X IncrA(X obj) {
- X temp = new X();
- temp.Val = obj.Val + 1;
- return temp;
- }

```
// This method returns Y and has a Y parameter.  
static Y IncrB(Y obj) {  
    Y temp = new Y();  
    temp.Val = obj.Val + 1;  
    return temp;  
}
```

```
static void Main() {  
    Y Yob = new Y();  
    // In this case, the parameter to IncrA  
    // is X and the parameter to Changelt is Y.  
    // Because of contravariance, the following  
    // line is OK.  
    Changelt change = IncrA;  
    X Xob = change(Yob);  
    Console.WriteLine("Xob: " + Xob.Val);  
    // In the next case, the return type of  
    // IncrB is Y and the return type of  
    // Changelt is X. Because of covariance,  
    // the following line is OK.  
    change = IncrB;  
    Yob = (Y) change(Yob);  
    Console.WriteLine("Yob: " + Yob.Val);  
}  
}
```


- The output from the program is shown here:

Xob: 1

Yob: 1

- In the program, notice that class Y is derived from class X.
- Next, notice that the delegate Changelt() is declared like this:
 delegate X Changelt(Y obj);
- Changelt() returns X and has a Y parameter.
- Next, notice that the methods IncrA() and IncrB() are declared as shown here:
 static X IncrA(X obj)
 static Y IncrB(Y obj)

- The `IncrA( )` method has an `X` parameter and returns `X`.
- The `IncrB( )` method has a `Y` parameter and returns `Y`.
- Given covariance and contravariance, either of these methods can be passed to `Changelt`, as the program illustrates.

- Therefore, this line
 `Changelt change = IncrA;`
- uses contravariance to enable `IncrA( )` to be passed to the delegate because `IncrA( )` has an `X` parameter, but the delegate has a `Y` parameter.
- This works because, with contravariance, if the parameter type of the method passed to a delegate is a base class of the parameter type used by the delegate, then the method and the delegate are compatible.

- The next line is also legal, but this time it is because of covariance:
 `change = IncrB;`
- In this case, the return type of `IncrB( )` is `Y`, but the return type of `Changelt( )` is `X`.
- However, because the return type of the method is a class derived from the return type of the delegate, the two are compatible.

System.Delegate

- All delegates are classes that are implicitly derived from **System.Delegate**.

Why Delegates

- In general, delegates are useful for two main reasons.
- First, as shown later, delegates support events.
- Second, delegates give the program a way to execute methods at runtime without having to know precisely what those methods are at compile time.
- This ability is quite useful when we want to create a framework that allows components to be plugged in.
- For example, imagine a drawing program (a bit like the standard Windows Paint accessory).
- Using a delegate, we could allow the user to plug in special color filters or image analyzers.
- Furthermore, the user could create a sequence of these filters or analyzers.
- Such a scheme could be easily handled using a delegate.

Anonymous Function

- We will often find that the method referred to by a delegate is used only for that purpose.
- In other words, the only reason for the method is so it can be invoked via a delegate.
- The method is never called on its own.
- In such a case, we can avoid the need to create a separate method by using an *anonymous function*.

- *An anonymous function is, essentially, an unnamed block of code that is passed to a delegate constructor.*
- One advantage to using an anonymous function is simplicity.
- There is no need to declare a separate method whose only purpose is to be passed to a delegate.

Anonymous Methods

- An anonymous method is one way to create an unnamed block of code that is associated with a specific delegate instance.
- An anonymous method is created by following the keyword **delegate** with a block of code.

The following program uses an anonymous method that counts from 0 to 5.

```
// Demonstrate an anonymous method.
using System;
// Declare a delegate type.
delegate void CountIt();
class AnonMethDemo {
static void Main() {
// Here, the code for counting is passed
// as an anonymous method.
CountIt count = delegate {
// This is the block of code passed to the delegate.
for(int i=0; i <= 5; i++)
Console.WriteLine(i);
}; // notice the semicolon
count();
}
}
```

- This program first declares a delegate type called `CountIt` that has no parameters and returns `void`.
- Inside `Main( )`, a `CountIt` instance called `count` is created, and it is passed the block of code that follows the `delegate` keyword.
- This block of code is the anonymous method that will be executed when `count` is called.
- Notice that the block of code is followed by a semicolon, which terminates the declaration statement.

- The output from the program is shown here:

0

1

2

3

4

5

Pass Arguments to an Anonymous Method

- It is possible to pass one or more arguments to an anonymous method.
- To do so, follow the delegate keyword with a parenthesized parameter list.
- Then, pass the argument(s) to the delegate instance when it is called.
- For example, here is the preceding program rewritten so that the ending value for the count is passed:

```
// Demonstrate an anonymous method that takes an argument.
using System;
// Notice that CountIt now has a parameter.
delegate void CountIt(int end);
class AnonMethDemo2 {
static void Main() {
// Here, the ending value for the count
// is passed to the anonymous method.
CountIt count = delegate (int end) {
for(int i=0; i <= end; i++)
Console.WriteLine(i);
};
count(3);
Console.WriteLine();
count(5);
}
}
```

- In this version, CountIt now takes an integer argument.
- Notice how the parameter list is specified after the delegate keyword when the anonymous method is created.
- The code inside the anonymous method has access to the parameter and in just the same way it would if a named method were being created.
- The output from this program is shown next:

0
1
2
3
0
1
2
3
4
5

Return a Value from an Anonymous Method

- An anonymous method can return a value.
- The value is returned by use of the **return** statement, which works the same in an anonymous method as it does in a named method.
- The type of the return value must be compatible with the return type specified by the delegate.
- For example, here the code that performs the count also computes the summation of the count and returns the result:

```
// Demonstrate an anonymous method that returns a value.
using System;
// This delegate returns a value.
delegate int CountIt(int end);
class AnonMethDemo3 {
static void Main() {
int result;
// Here, the ending value for the count
// is passed to the anonymous method.
// A summation of the count is returned.
CountIt count = delegate (int end) {
int sum = 0;
for(int i=0; i <= end; i++) {
Console.WriteLine(i);
sum += i;
}
return sum; // return a value from an anonymous method
};
```

```
result = count(3);  
Console.WriteLine("Summation of 3 is " + result);  
Console.WriteLine();  
result = count(5);  
Console.WriteLine("Summation of 5 is " + result);  
}  
}
```

- In this version, the value of sum is returned by the code block that is associated with the count delegate instance.
- Notice that the return statement is used in an anonymous method in just the same way that it is used in a named method.
- The output is shown here:

0

1

2

3

Summation of 3 is 6

0

1

2

3

4

5

Summation of 5 is 15

Use Outer Variables with Anonymous Methods

- A local variable or parameter whose scope includes an anonymous method is called an *outer variable*.
- *An anonymous method has access to and can use these outer variables.*
- *When an outer variable is used by an anonymous method, that variable is said to be captured.*
- A captured variable will stay in existence at least until the delegate that captured it is subject to garbage collection.
- Thus, even though a local variable will normally cease to exist when its block is exited, if that local variable is being used by an anonymous method, then that variable will stay in existence at least until the delegate referring to that method is destroyed.

- The capturing of a local variable can lead to unexpected results.
- For example, consider this version of the counting program.

```
// Demonstrate a captured variable.  
using System;  
// This delegate returns int and takes an int argument.  
delegate int CountIt(int end);  
class VarCapture {  
    static CountIt Counter() {  
        int sum = 0;  
        // Here, a summation of the count is stored  
        // in the captured variable sum.  
        CountIt ctObj = delegate (int end) {  
            for(int i=0; i <= end; i++) {  
                Console.WriteLine(i);  
                sum += i;  
            }  
            return sum;  
        };  
        return ctObj;  
    }  
}
```

```
static void Main() {  
    // Get a counter.  
    CountIt count = Counter();  
    int result;  
    result = count(3);  
    Console.WriteLine("Summation of 3 is " + result);  
    Console.WriteLine();  
    result = count(5);  
    Console.WriteLine("Summation of 5 is " + result);  
}  
}
```


- The output is shown here. Pay special attention to the summation value.
- 0
- 1
- 2
- 3
- Summation of 3 is 6
- 0
- 1
- 2
- 3
- 4
- 5
- Summation of 5 is 21

- As in the previous version, the summation of the count is computed.
- However, in this version, a CountIt object is constructed and returned by a static method called Counter().
- This object uses the variable sum, which is declared in the enclosing scope provided by Counter(), rather than in the anonymous method, itself.
- Thus, sum is captured by the anonymous method.
- Inside Main(), Counter() is called to obtain a CountIt object.
- Thus, sum will not be destroyed until the program finishes.

- The count still proceeds normally.
- However, notice the summation value for 5.
- It shows 21 instead of 15!
- The reason for this is that sum is captured by ctObj when it is created by the Counter() method.
- This means it remains in existence until count is subject to garbage collection at the end of the program.
- Thus, its value is not destroyed when Counter() returns or with each call to the anonymous method when count is called in Main().

- Although captured variables can result in rather counterintuitive situations, such as the one just shown, it makes sense if you think about it a bit.
- The key point is that when an anonymous method captures a variable, that variable cannot go out of existence until the delegate that captures it is no longer being used.
- If this were not the case, then the captured variable could be undefined when it is needed by the delegate.

Invocation List

- Multicast delegates maintain an invocation list, which has an entry for each delegate of the multicast delegate.
- Entries are added to the invocation list in the same order in which delegates are added.
- *GetInvocationList* returns the invocation list as an array of delegates.

- This is the syntax of the *GetInvocationList* method:
- `delegate [] GetInvocationList()`
- This code retrieves the list of delegates in a multicast delegate, in which each delegate is a wrapper of a function.
- The name of each function is displayed.
- The *Delegate.Method* property returns a *MethodInfo* type, which encapsulates the function abstracted by the delegate.

```
using System;
Namespace CSharp{
public delegate void DelegateClass();
public class Starter{ public static void Main(){
DelegateClass del=(DelegateClass)
DelegateClass.Combine(new DelegateClass []
{ MethodA, MethodB, MethodA, MethodB } );
del();
foreach(DelegateClass item in del.GetInvocationList()) {
Console.WriteLine(item.Method.Name+ " in invocation list.");
}
}
public static void MethodA() {
Console.WriteLine("MethodA...");
}
public static void MethodB() {
Console.WriteLine("MethodB...");
} } }
```

- The delegate signature can contain reference parameters.
- When invoked, the reference is passed to the called functions.
- Reference parameters are a way for functions on an invocation list to share state.
- Value parameters are not shareable across the functions.
- Because the invocation list is called in sequence, beginning with the first delegate and function, each successive entry in the chain can view changes in the reference parameter.

- The next item in the chain can view those changes and possibly change the state again.
- In this way, when the multicast delegate is invoked, the state information is propagated along the invocation list.
- Furthermore, functions in invocation lists that have the same target object or class share the state of the object or class.

- The following code (Next slide) uses the reference parameter of a multicast delegate as a counter.
- The delegate has a value and reference parameter.
- The value parameter is lost between function calls in the invocation list.
- However, the reference parameter persists

```
using System;
Namespace Csharp{
    public delegate void DelegateClass(int valCount, ref int refCount);
    public class Counter{ public static void Main(){
        DelegateClass del=(DelegateClass) AddOne+ (DelegateClass) AddTwo+
            (DelegateClass) AddOne;
        int valCount=0;
        int refCount=0;
        del(valCount, ref refCount);
        Console.WriteLine("Value count = {0}", valCount); // 0
        Console.WriteLine("Reference count = {0}", refCount); // 4
    }
    public static void AddOne(int valCount, ref int refCount)
    {
        ++valCount;
        ++refCount;
    }
    public static void AddTwo(int valCount, ref int refCount)
    {
        valCount+=2;
        refCount+=2;
    } } }
```

Methods and properties

- Several members of delegate object, such as *GetInvocationList*, have already been introduced.
- Table (Next slide) is a list of some of the public properties and methods of the *MulticastDelegate* type.
- Some of the members are inherited from *System.Delegate*.

Member	Description
<i>BeginInvoke</i>	Invokes a delegate asynchronously.
<i>Combine</i>	Combines delegates into a multicast delegate. This is a static method.
<i>CreateDelegate</i>	Defines a delegate at run time. This is a static method.
<i>DynamicInvoke</i>	Dynamically invokes a delegate that was created at run time.
<i>EndInvoke</i>	Requests the results of a delegate that was executed asynchronously.
<i>Invoke</i>	Executes a delegate, which calls all functions contained in the delegate.
<i>Method</i>	This is a property that returns the <i>MethodInfo</i> type of the last function in the invocation list. <i>MethodInfo</i> provides a description of a function.
<i>Remove</i>	Removes a delegate from a multicast delegate. This is a static method.
<i>RemoveAll</i>	Removes the invocation list of a delegate from the invocation list of another delegate. The result is returned. This is a static method.
<i>Target</i>	This is a property that returns the instance of the last function in the invocation list. For static functions, <i>Target</i> is null.

Generics and Delegates

- A delegate can contain closed generic methods.
 - A generic type or method is closed if instantiable types have been substituted for all its type parameters, including all the type parameters of all enclosing types.
- Generic method must be compatible with the signature of the delegate

- A [delegate](#) can define its own type parameters.
- Code that references the generic delegate can specify the type argument to create a closed constructed type, just like when instantiating a generic class or calling a generic method, as shown in the following example:

```
public delegate void Del<T>(T item);  
public static void Notify(int i) { }  
Del<int> m1 = new Del<int>(Notify);
```

Asynchronous Invocation

- The .NET Framework enables you to call any method asynchronously.
- The big advantage with .NET asynchronous method invocation is that we can take any method we have in our project, and we can call it asynchronously without touching the code of our method.
- This is done by defining a delegate with the same signature as the method we want to call.
- The common language runtime automatically defines `BeginInvoke` and `EndInvoke` methods for this delegate, with the appropriate signatures.

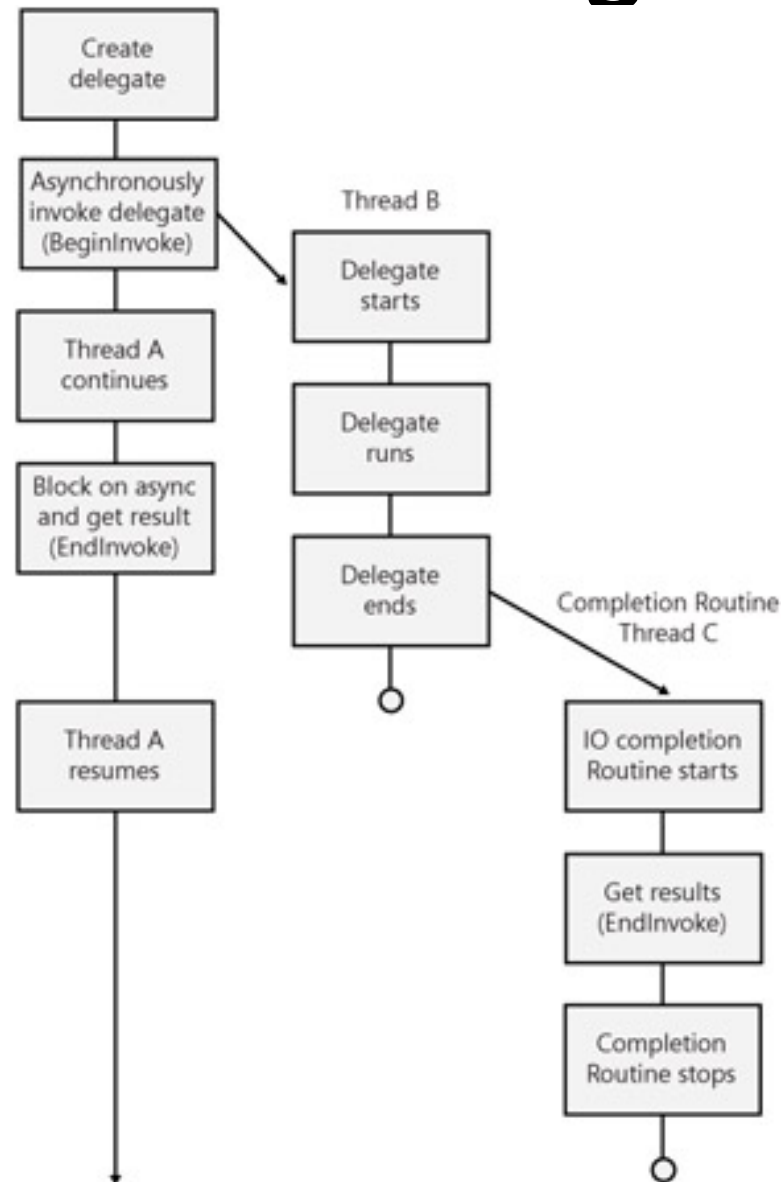
- The BeginInvoke method initiates the asynchronous call.
- It has the same parameters as the method that we want to execute asynchronously, plus two additional optional parameters.
- The first parameter is an [AsyncCallback](#) delegate that references a method to be called when the asynchronous call completes.
- The second parameter is a user-defined object that passes information into the callback method.
- BeginInvoke returns immediately and does not wait for the asynchronous call to complete.
- BeginInvoke returns an [IAsyncResult](#), which can be used to monitor the progress of the asynchronous call.

- The EndInvoke method retrieves the results of the asynchronous call.
- It can be called any time after BeginInvoke.
- If the asynchronous call has not completed, EndInvoke blocks the calling thread until it completes.
- The parameters of EndInvoke include the out and ref parameters of the method that you want to execute asynchronously, plus the [IAsyncResult](#) returned by BeginInvoke.

AsyncResult Properties

Property	Description
<i>AsyncState</i>	The state object that has information on the asynchronous operation.
<i>AsyncWaitHandle</i>	Use the <i>WaitHandle</i> to block until the asynchronous operation is completed.
<i>CompleteSynchronously</i>	Indicates that the operation completed synchronously.
<i>IsCompleted</i>	Indicates that the asynchronous operation has completed.

Asynchronous Delegate Diagram



Exceptions

- What happens if an unhandled exception is raised in a multicast delegate?
- If the delegate is invoked inside a protected block of code, the exception is trapped.
- The invocation list is called until an exception occurs.
- Functions later in the invocation list do not execute.

- Let us assume that the invocation list has pointers to *MethodA*, *MethodB*, *MethodC*, and *MethodD*.
- They were also added to the delegate in that order.
- If an unhandled exception is raised in *MethodB*, *MethodC* and *MethodD* are not called.

- The situation might appear to be more complicated if the exception is raised in a delegate that is running asynchronously.
- In this circumstance, the exception is raised on a different thread.
- However, the run time will route the exception back to the calling thread, as evidenced in the following code.
- Therefore, the exception is handled as if the delegate were called synchronously.

```
using System;
using System.Threading;
Namespace CSharpBook{
public delegate void DelegateClass();
public class Starter{
public static void Main(){
Console.WriteLine("Running on primary thread");
try { DelegateClass del=MethodA;
IAsyncResult ar=del.BeginInvoke(null, null);
del.EndInvoke(ar);
}
catch(Exception except){
Console.WriteLine("Running on primary thread");
Console.WriteLine("Exception caught: " +except.Message);
}}
public static void MethodA() {
if(Thread.CurrentThread.IsThreadPoolThread==true)
{
Console.WriteLine( "Running on a thread pool thread");
}
else
{
Console.WriteLine("Running on primary thread");
}
throw new Exception("failure");
}
}}
```


Events

- Another important C# feature is built upon the foundation of delegates: the *event*.
- *An event* is, essentially, an automatic notification that some action has occurred.

- Events work like this:
- An object that has an interest in an event registers an event handler for that event.
- When the event occurs, all registered handlers are called.
- Event handlers are represented by delegates.
- Events are members of a class and are declared using the **event keyword**.

- Its most commonly used form is :
event event-delegate event-name;
- Here, *event-delegate* is the name of the delegate used to support the event,
- *event-name* is the name of the specific event object being declared.

```
// A very simple event demonstration.  
using System;  
// Declare a delegate type for an event.  
delegate void MyEventHandler();  
// Declare a class that contains an event.  
class MyEvent {  
    public event MyEventHandler SomeEvent;  
    // This is called to fire the event.  
    public void OnSomeEvent() {  
        if(SomeEvent != null)  
            SomeEvent();  
    }  
}
```

```
class EventDemo {  
    // An event handler.  
    static void Handler() {  
        Console.WriteLine("Event occurred");  
    }  
    static void Main() {  
        MyEvent evt = new MyEvent();  
        // Add Handler() to the event list.  
        evt.SomeEvent += Handler;  
        // Fire the event.  
        evt.OnSomeEvent();  
    }  
}
```

- This program displays the following output:
Event occurred

- Although simple, this program contains all the elements essential to proper event handling.
- The program begins by declaring a delegate type for the event handler, as shown here:
delegate void MyEventHandler();

- All events are activated through a delegate.
- Thus, the event delegate type defines the return type and signature for the event.
- In this case, there are no parameters, but event parameters are allowed.

- Next, an event class, called MyEvent, is created.
- Inside the class, an event called SomeEvent is declared, using this line:

public event MyEventHandler SomeEvent;

- The keyword `event` tells the compiler that an event is being declared.
- Also declared inside `MyEvent` is the method `OnSomeEvent( )`, which is the method a program will call to signal (or “fire”) an event.
- (That is, this is the method called when the event occurs.)
- It calls an event handler through the `SomeEvent` delegate, as shown here:
`if(SomeEvent != null)`
`SomeEvent();`

- Notice that a handler is called if and only if `SomeEvent` is not null.
- Since other parts of the program must register an interest in an event in order to receive event notifications, it is possible that `OnSomeEvent( )` could be called before any event handler has been registered.
- To prevent calling on a null reference, the event delegate must be tested to ensure that it is not null.

- Inside **EventDemo**, an event handler called **Handler()** is created.
- **In this simple example**, the event handler simply displays a message, but other handlers could perform more meaningful actions.
- In **Main()**, a **MyEvent** object is created, and **Handler()** is registered as a handler for this event, by adding it as shown here:

```
MyEvent evt = new MyEvent();  
// Add Handler() to the event list.  
evt.SomeEvent += Handler;
```

- Notice that the handler is added using the **`+=` operator**.
- **Events support only `+=` and `-=`.**
- **In this case, `Handler( )` is a static method, but event handlers can also be instance methods.**

- Finally, the event is fired as shown here:

// Fire the event.

evt.OnSomeEvent();

- Calling OnSomeEvent() causes all registered event handlers to be called.
- In this case, there is only one registered handler, but there could be more.

A Multicast Event Example

- Like delegates, events can be multicast.
- This enables multiple objects to respond to an event notification.


```
// An event multicast demonstration.  
using System;  
// Declare a delegate type for an event.  
delegate void MyEventHandler();  
// Declare a class that contains an event.  
class MyEvent {  
    public event MyEventHandler SomeEvent;  
    // This is called to fire the event.  
    public void OnSomeEvent() {  
        if(SomeEvent != null)  
            SomeEvent();  
    }  
}
```

```
class X {  
    public void Xhandler() {  
        Console.WriteLine("Event received by X  
            object");  
    }  
}  
  
class Y {  
    public void Yhandler() {  
        Console.WriteLine("Event received by Y object");  
    }  
}
```

```
class EventDemo2 {  
    static void Handler() {  
        Console.WriteLine("Event received by  
            EventDemo");  
    }  
    static void Main() {  
        MyEvent evt = new MyEvent();  
        X xOb = new X();  
        Y yOb = new Y();  
    }  
}
```

```
// Add handlers to the event list.  
evt.SomeEvent += Handler;  
evt.SomeEvent += xOb.Xhandler;  
evt.SomeEvent += yOb.Yhandler;  
// Fire the event.  
evt.OnSomeEvent();  
Console.WriteLine();  
// Remove a handler.  
evt.SomeEvent -= xOb.Xhandler;  
evt.OnSomeEvent();  
}  
}
```

- The output from the program is shown here:
- Event received by EventDemo
- Event received by X object
- Event received by Y object
- Event received by EventDemo
- Event received by Y object

- This example creates two additional classes, called **X and Y, which also define event** handlers compatible with **MyEventHandler**.
- **Thus, these handlers can also become part** of the event chain.
- Notice that the handlers in **X and Y are not static**.
- **This means that objects** of each must be created, and the handler linked to each instance must be added to the event chain.

Events

- An event is an occurrence that an object or class wants to notify someone else about.
 - (The *someone else* is either another object or class.)
- A button notifies a form object that someone clicked on the button.
- A timer notifies some other object that a time period has elapsed.
- An application domain notifies an application of an unhandled event.
- A Web application notifies an object of an application event.

- We can create custom events, such as a device notifying a monitoring application that data transmission has started or stopped.
- Events are literally anything worth notifying someone else about.
- Public events are exposed as public members of a publisher class, such as a button class.
- We can also have private and protected events that are used within the realm of the containing or derived class

- Any object or class interested in an event can subscribe.
- Subscribers register for an event by submitting a delegate.
- The delegate must be single cast and contain a single function pointer.
- That function is the subscriber's response to the event.

- When the event is raised, the publisher calls the function, affording the subscriber an opportunity to respond to the event.
- For this reason, the function is called an event handler.
- Events can have multiple subscribers.
- A device could publish a power-off event in which multiple applications might want to be notified and respond.

- What happens when there are no subscribers to a particular event?
 - Events that have no subscribers are not raised.

Publishing an Event

- Both classes and objects can publish events. Static events are published at classes, whereas objects publish instance events.
- Multicast delegates underlie events.
- We must create a delegate for the event or leverage a predefined event.
- Events are defined with the event keyword. This is the syntax for defining an event:
 - ***accessibility event delegatename eventname***

- Events are defined as a field in a class.
- The accessibility of an event is typically public or protected.
- Protected events restrict subscribers to child objects.
- A public event is available to every interested subscriber.
- Delegate name is the underlying delegate of the event, which defines the signature of the event.
- Event name is the identity of the event.

```
public delegate void DelegateClass();  
public event DelegateClass MyEvent;
```

- As a best practice, event handlers should return void and have two arguments.
- *EventHandler* is a predefined delegate created for this purpose.
- It is included in the .NET Framework class library (FCL) and prevents from having to declare a separate delegate for the standard event type.
- The first parameter is the object that raised the event.
- The second parameter is an *EventArgs* derived instance, which offers optional data that further explains the event.

- Internally, events declared in a class become a field of that class.
- Delegates used to define the event, which is a multicast delegate, are types used for the field.
- The C# compiler also inserts methods to add and remove subscribers to the event.
- The names of the methods are *add_EventName* and *remove_EventName*, respectively.
- The fourth member added to the class is the event itself.

Subscriber

- The publisher/subscriber relationship is a *one-to-many* relationship.
- For each publisher there can be zero or more subscribers.
- Conversely, a subscriber can subscribe to multiple events.
 - For example, a form can subscribe to a button click and a text change event.

- Subscribers add a delegate to an event to register for the event.
- The delegate is a wrapper for the function to be called when the event is raised.

Subscribe to an event using the add method or the += assignment operator:

```
using System;
Namespace CSharp{
class Publisher {
public event EventHandler MyEvent;   }
public class Subscriber{
public static void Handler(object obj, EventArgs args) {   }
public static void Main(){
    Publisher pub=new Publisher();
    pub.MyEvent+=Handler; // other processing
} } }
```

- Cancel a subscription to a delegate with the -= assignment operator.

Raising an Event

- At the discretion of a publisher, an event is raised and the methods of the subscribers are called back.
- Raise an event with the call operator "()".
- Adding the call operator to the event raises the event.

- Because a delegate underlies an event, the *Invoke* method inherited from a multicast delegate is another means to raising an event.
- The signature of both the call operator and the *Invoke* method match the delegate of the event.
- Any parameters are passed to the called methods of the subscribers

- If an event returns a value as defined by the event delegate, the function of the last subscriber sets the return.
- Do not raise an event for events without subscribers.
- Events with no subscribers are null, which can be tested.
- An application that raises an event that has no subscribers will incur an exception because of a null reference.

- The proper way to raise an event:

```
public void SomeMethod()  
{  
    if(anEvent!=null)  
    {  
        anEvent(null, null);  
    }  
}
```

EventArgs

- Events sometime provide subscribers with custom information pertaining to the event.
- The mouse click event provides the *x* and *y* coordinates of the mouse pointer in the *MouseEventArgs* class.
- The *DataRowChangeEventArgs* class provides the affected row and action of several database-related events, such as the *RowChanged* event.

- The *PaintEventArgs* class is instantiated in a paint event.
- It gives developers the clip rectangle and graphics object for the paint event.
- Custom information about an event is defined in an *EventArgs* derived class.
- *MouseEventArgs*, *DataRowChangeEventArgs*, *PaintEventArgs*, and other related classes derive from the *EventArgs* class.
- These classes represent the state information for the event.

- The *EventArgs* derived class that contains the state information of the event is typically passed as the second parameter of the event.
- State information can also be specified in the other parameters of events.

Code for a bank account

- The *NSF* event is raised when the account is overdrawn.
- The *BankEventArgs* class provides the bank account balance and amount of the transaction that would overdraw the account.

Code for a bank account

```
using System;
namespace CSharp{
    public class Starter {
    public static void Main(){
        Bank account=new Bank();
        account.NSF+=NSFHandler;
        account.Deposit(500);
        account.Withdrawal(750);
    }
    public static void NSFHandler(object o, BankEventArgs e) {
    Console.WriteLine("NSF Transaction");
    Console.WriteLine("Balance: {0}", e.Balance);
        Console.WriteLine("Transaction: {0}", e.Transaction);
    } }
}
```

```
public delegate void OverDrawn(object o,  
    BankEventArgs e);  
public class Bank  
{  
    public event OverDrawn NSF; // non sufficient funds  
    public decimal Deposit(decimal amountDeposit)  
    {  
        propBalance+=amountDeposit;  
        return propBalance;  
    }  
}
```

```
public decimal Withdrawal(decimal amountWithdrawn)
{
    decimal newBalance=propBalance-amountWithdrawn;
    if(newBalance < 0)
    {
        if(NSF != null)
        {
            BankEventArgs args=new BankEventArgs( Balance,
                amountWithdrawn);
            NSF(this, args);
        }
        return propBalance;
    }
    return propBalance=newBalance;
}
```

```
private decimal propBalance=0;  
public decimal Balance  
{  
    get  
{  
    return propBalance;  
}  
}  
}
```

```
public class BankEventArgs: EventArgs {
    public BankEventArgs(decimal amountBalance, decimal amountTransaction)
    {
        propBalance=amountBalance;
        propTransaction=amountTransaction;
    }
    private decimal propBalance;
    public decimal Balance {
        get {
            return propBalance;
        }
    }
    private decimal propTransaction;
    public decimal Transaction {
        get {
            return propTransaction;
        }
    }
} }
```